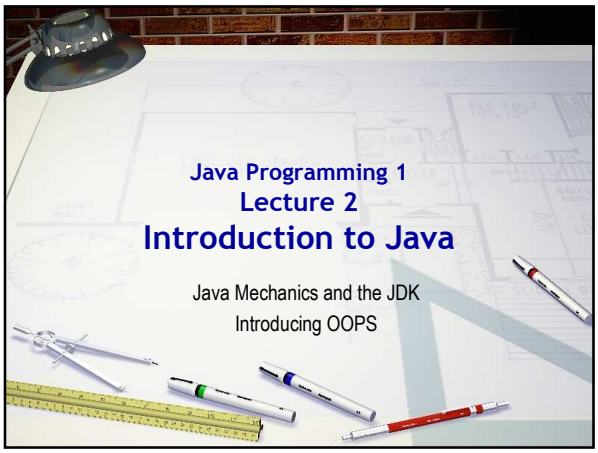


Installing the Java JDK

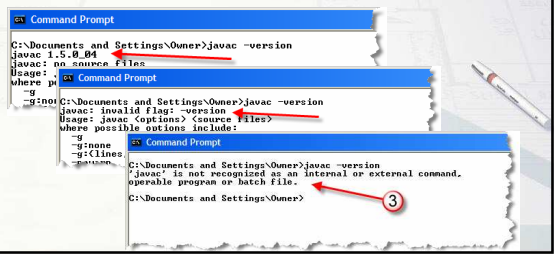
- If you want to work at home, you'll need the Java Standard Edition, Java Development Kit (J2SE JDK)
- Complete video walkthroughs on my web site:
  - http://csjava.occ.cccd.edu/~gilberts/JDK6-install/
  - http://csjava.occ.cccd.edu/~gilberts/java6JDK/
  - http://csjava.occ.cccd.edu/~gilberts/JDK6/
- Visit http://java.sun.com - click on J2SE link
- Set the PATH environmental variable to include JDK **bin** folder where you installed the JDK

Step 0 - Your Installation

- Open OpenOffice Writer and create a new document
- Save the file as LastnameFirstname_ic02
  - Replace LastnameFirstname with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 2"
- **Exercise 1**: Open a command shell
  - Describe what you did to open it
  - Paste a picture of the shell window below description
- **Exercise 2**: Type **javac –version**
  - Describe the first line of output

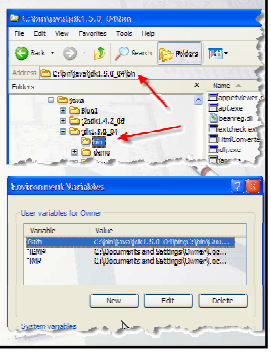
Trouble-shooting

- Outcome 1: All OK
- Outcome 2: Earlier JDK – need at least version 1.5
- Outcome 3: No JDK or bad path



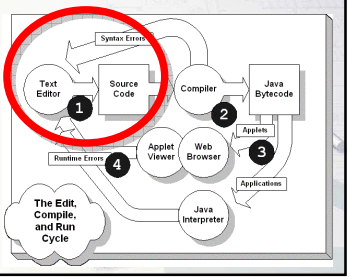
Fixing the Path

- **Step 1**: Locate the JDK bin folder in Windows Explorer
- **Step 2**: Open the Advanced Tab in My Computer, click Environmental Variables
- **Step 3**: find Path in User Variables, add JDK path
  - Not found: **New** to create
  - **Edit** to add to existing
- **Vista**: in User Accounts



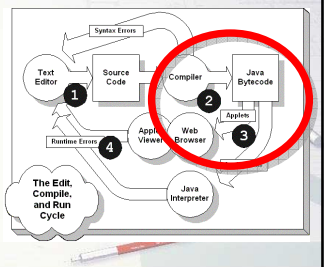
Step 1: Writing the Code

- Use a text editor to write your programming instructions in the Java language
  - Notepad, SciTE, vi, TextWrangler
- This is **source code**
- Save in a file with the extension **.java**



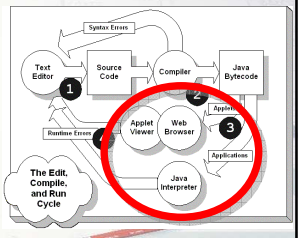
Step 2: Compiling

- Use a Java **compiler** to translate your source code into executable machine code
  - javac, jikes, Eclipse
- This is **bytecode**
- Produces a file with the extension **.class**
- If **syntax errors** occur, return to text editor



Step 3: Run your Program

- Use a Java **interpreter** or JIT compiler to translate the bytecode into native machine code
- Applications: **java**, **javaw**
- Applets: HTML
  - Web browser, appletviewer
- If **runtime errors** occur, return to text editor

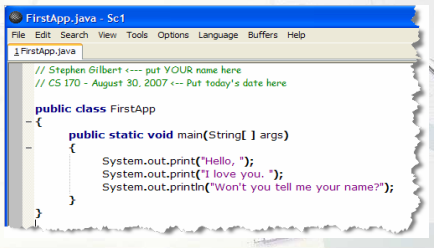


Your First Java Program

- We'll first write an **application** using the J2SE JDK
- The basic tools you'll need are:
  - A **text editor** to write your source code (we'll use **SciTE**)
  - The **javac compiler** and **java interpreter** from the JDK
- Using the command line
  - Create a **cs170home** folder in your **U:** drive
  - Create a **unit01** folder **inside** of cs170home
  - Start the SciTE editor (Sc1.exe) from the command line
- **Exercise 3**: describe steps you followed to do the above

Enter Java Source

- Save empty file inside **unit01** as : **FirstApp.java**
- Type and save (again) the following Java program
  - Note that "println" is short for "print line"; "el-en" not "1-en"



Compile your code

- **Exercise 4**: in the command window change to the directory containing your source code and list the files
  - What commands did you use?
  - Take a picture of listing and paste it under the answer
- Use the **javac** command to compile your code

```
U:\cs170home\unit01\> javac FirstApp.java
```

When Bad Things Happen

- Compiler Errors
  - Capitalization: (System not system, String not string)
  - Spelling the name of your source file incorrectly
  - Forgetting or mismatching the number of braces
  - Mistaking parentheses for braces
  - Misspelling the name of a variable
- Edit and then recompile until there are no errors
- **Exercise 5**: Display a list of files in your folder
  - What files does the folder contain after compiling?
  - Take a picture of listing and paste it under the answer

Run the program

- **Run** the program with this command:

```
U:\cs170home\unit01\> java FirstApp
```

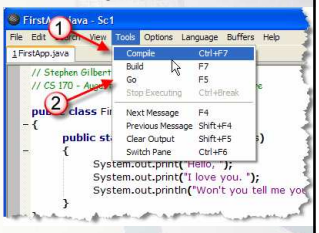
  - Note that you **don't** use the **.class** extension
  - Use the **java** interpreter, not the **javac** compiler
- **Exercise 6**: Show me the output of your program
  - Snap a screenshot or copy and paste the text
- What can go wrong at this point? 2 things:
  - Can't run without compile: **dir** to check the class file
  - The **CLASSPATH** environmental variable may be set

Messing with CLASSPATH

- Tells the Java interpreter where to look for classes
- Some programs (Kodak, QuickTime) set it incorrectly
- Three possible remedies:
  - 1. Type **set classpath=** every time you open a shell
  - 2. Use **java –cp .; ClassName** to run your files
  - 3. Add **.;** to the **classpath** environmental variable

Compiling From SciTE

- You can compile and run Java programs from **SciTE**
- To **compile**:
  - **Tools->Compile** from menu, or press **Ctrl+F7**
- To **run**:
  - Choose **Tools->Go** from the menu, or press **F5**



- **Exercise 7**: Compile and run, then take a screenshot

Correcting Errors in SciTE

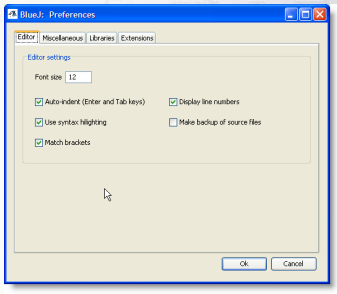
- **SciTE** will point out your **syntax** errors
  - Mistakes in program's "grammar" (missing semicolon)
  - Won't point out all errors though (logic and runtime)
- **Exercise 8a-e**: Make these changes to your program and compile. If the program compiles, then run it. Explain what happens each time. If necessary, show me a screenshot.
  - Change **main** to **Main**. Compiles? Runs?
  - Change **args** to **Args**.
  - Change **String** to **string**.
  - Change **public** to **Public**.
  - Change the first **{** to **(**

The BlueJ IDE

- A free Integrated Development Environment (IDE)
  - Written in Java, so it works on the Mac, Windows, Unix
    - Sma
    - Must
  - Integra gger
  - Adds a ava snippets
- Object-fo ed
  - Interac cts
- OCC versi apps\bluej

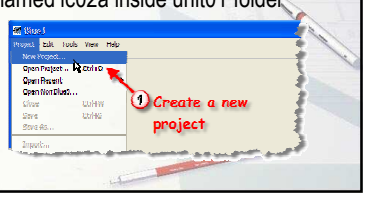
Configure

- Start from **Programming** folder on the desktop
- Configure using **Preferences** on the **Tools** menu
- Editor options
  - Line numbers
  - Font size
  - Backups
- **Exercise 9**: Snap a screenshot



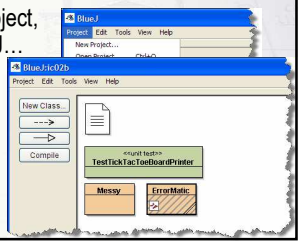
BlueJ Projects

- Put your programs into projects in BlueJ
  - Folders with source code and compiler information
  - Usually create one project for each program
  - Put projects in your cs170home folder
- Create a project named ic02a inside unit01 folder
- **Exercise 10**: shoot a screenshot



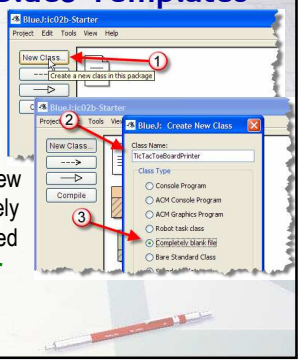
Importing Projects

- I'll supply starter code for many of your assignments and exercises. You must **import** to work with them
- Copy ic02b-Starter.zip from Q: to U:\cs170home
- To import an existing project, Choose Open Non-BlueJ… from Project menu
- **Exercise 11**: Locate the file ic02b-Starter.zip and open it.



BlueJ Templates

- BlueJ uses **templates** to get you started
  - To learn Java syntax, start with a blank file
- **Exercise 12**: Create a new class using the Completely Blank File template named **TicTacToeBoardPrinter**



Program Organization

- Every Java program is a **class definition**
  - Class definition is a blueprint to construct objects
  - The basic class structure looks like this:

```
... some stuff
public class SomeClassName
{
   ... some more stuff
}
```

  - Must be stored in **SomeClassName.java**
  - Only one **public** class per file

The Class Heading

- Class definitions have two parts: **heading** & **body**
  - Heading for classes looks like this:

```
1: public class FirstApp
2: {
```

- Starts with a **declaration**: **public class**
  - Announces to the compiler what is to follow
- Both of these are **keywords** or **reserved words**.
  - "Built-in" vocabulary (50 words, page 1121-23 (App F)
  - Always in lowercase
  - Can't be used for variables or methods
- **Exercise 13**: identify each of the keywords in FirstApp

The Name of the Class

- The name of the class is: **FirstApp**
  - This is an **identifier**, not a keyword
  - An identifier is a name that **you** make up
    - You'll create names for classes, methods, fields, etc.
- Java language **syntax rules** for identifiers
  - Case sensitive
  - Contains letters, digits, $, and underscore
    - Avoid using the $, Java uses it internally
  - Cannot start with a digit
  - Cannot use a reserved or keyword as an identifier
- **Exercise 14**: list all of the **identifiers** used in FirstApp

Express Yourself

- **Exercise 15**: As we look at different syntax issues we'll complete the **TicTacToeBoardPrinter** program on page 30 in your book (Exercise P1.3)
  - Double-click TicTacToeBoardPrinter to open it
  - Add the necessary code to create the class
  - Compile it using the Compile button in the editor
  - Click the Run Tests button
  - Shoot a screenshot with the first test running correctly.

Method Basics

- Class body is delimited by braces **{ }** (begin...end)
- Inside the class body, you'll define two things:
  - **Attributes** -- the data fields used to store object state
  - **Methods** -- where the actions take place in an object
- Methods are used in two places in your programs
  - When you **define** or specify instructions to perform
  - When you **use**, **invoke** or **call** a method

Defining a Method

- An object's methods are where actions occur
  - The **main()** method is the application "starting point"
- Methods are **defined inside** your class
  - Consist of a heading and body, just like a class

```
4  public class FirstApp
5  {
6     public static void main(String[ ] args)
7     {
8        System.out.print("Hello, ");
9        System.out.print("I love you. ");
10       System.out.println("Won't you tell me your name?");
11    }
12 }
```

The Method Heading

- This method heading contains these five pieces
  - **public** : **access specifier** determines **who** can invoke it. If **public**, anyone, if **private**, only other methods in class.
  - **static** : **modifier**. Method is associated with the class. Most methods will **not** use this modifier, but **main()** **must**
  - **void** : **method return type**. For procedures, put **void** here.
  - **main** : the **name of the method**, used to invoke it. Java looks for this name when it runs your program
  - **(String[] args)** : the **parameter list**, consisting of a set of parentheses and information used to customize its behavior.

The Method Body

- Delimited by braces **{ }** (begin...end) like a class
- Inside the body you'll put Java **statements**
  - Think of a sentence in a natural language
  - Every statement **ends** with a **semicolon ;**
- The **FirstApp main()** method has **three** statements:
  - Prints a phrase on the console window when run

```
4  public class FirstApp
5  {
6      public static void main(String[ ] args)
7      {
8          System.out.print("Hello, ");
9          System.out.print("I love you. ");
10         System.out.println("Won't you tell me your name?");
11     }
12 }
```

Express Yourself

- **Exercise 16**: Add an **empty** **main()** method to your **TicTacToeBoardPrinter** program. (You won't have any statements inside your method, but you need to provide the correct heading and the braces for the method body.)
- Compile and then run the test program again. Keep correcting your program and re-running the tests until you've defined the **main()** method correctly, except for the program output. (The very last test).
  - Shoot a screen-shot showing the tests passing

Calling a Method

- A method is some code that performs an action
  - You "tell" the object to perform the action like this:

```
System.out.print("Hello, I love you.");
```

- This is often called "sending a message" to the object
  - The message is **print()**
  - Message is sent to a **PrintStream object** named **out**
    - This object "lives" inside the **System class**
  - The **String "Hello, I love you."** is a **parameter**
- Also called **invoking** or **calling** a method

Displaying Output

- The **System.out** object is known as the **console** or **standard output** object.
  - The single **parameter** (or **argument**) you supply is displayed in a text-based window
  - The parameter can be text (inside quotes) or a number

```
System.out.print("Hello, ");
System.out.println("I love you.");
```

- The **print()** method leaves the cursor on the same line, while **println()** moves it to the next line

Express Yourself

- **Exercise 17**: Use the console output object and its **print()** and/or **println()** methods to cause **TicTacToeBoardPrinter** to display a Tic Tac Toe board. Compile and then run the test program again. Keep correcting your program and re-running the tests until all the tests work correctly.
  - Shoot a screen-shot showing the tests passing

Style & Punctuation

- Style and Punctuation Issues
  - Java is **freeform** language; relies on punctuation, not position of characters on the line
  - Up to you to use space to add readability
    - Try to group related statements into "paragraphs"
    - Span lines if statement is large
    - Don't normally put multiple statements on one line
    - Indent statements inside class and method bodies
  - See **bad** example on page 17
- See the **CS 170 Programming Style Guide**

Express Yourself

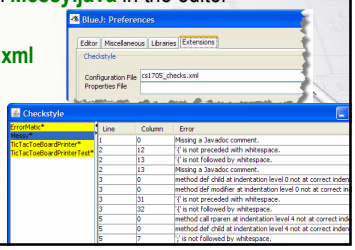
- **Exercise 18**: Use the console output object and its **print()** and/or **println()** methods to cause **TicTacToeBoardPrinter** to display a Tic Tac Toe board. Compile and then run the test program again. Keep correcting your program and re-running the tests until all the tests work correctly.
  - Shoot a screen-shot showing the tests passing
- **Exercise 19**: You've learned enough syntax to fix the **ErrorMatic** program. Get it to compile and run. Shoot me a screen-shot of the output.

Comments

- Comments help human readers understand your code
  - Comments are ignored by the compiler
- Java has three kinds of comments
  - **Single line**: start with **//**, end at the end of a line
  - **Multi-line** comments, delimited by **/*** and ***/**
  - **JavaDoc** comments, start with **/****, end with ***/**
    - Use special tags like **@author** and **@version**
    - See the style guide for more info on required tags

Checking Your Style

- BlueJ has a "plugin" that will check your style
  - Based on rules very similar to our style guide
- **Exercise 20**: Open **Messy.java** in the editor
  - Configure using **cs1705_checks.xml**
  - Choose Tools Checkstyle
  - Select Messy and correct



What is Web-Cat?

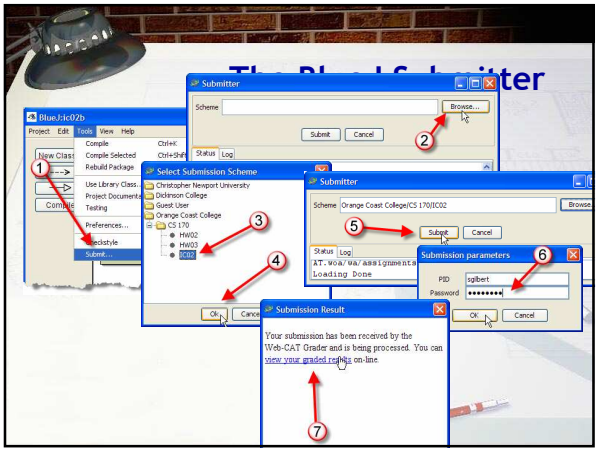
- An online homework submission and testing tool
  - Developed by Stephen Edwards at VT
  - Awards for helping students learn
  - Password sent to your OCC email
- Submit your assignments via the Web
  - Simple programs graded on how well they meet the tests that I develop similar to the one we just ran
  - Later, you'll supply your own tests

Simple Testing

- Testing is based on :
  - checking the syntax against the specification
  - running your code with different inputs
  - comparing your output with expected (correct) output
- Most homework assignments are graded
  - 4.5 points given for correct output
  - .5 point given for style using Checkstyle and PMD
    - Comments, indenting, following style guide

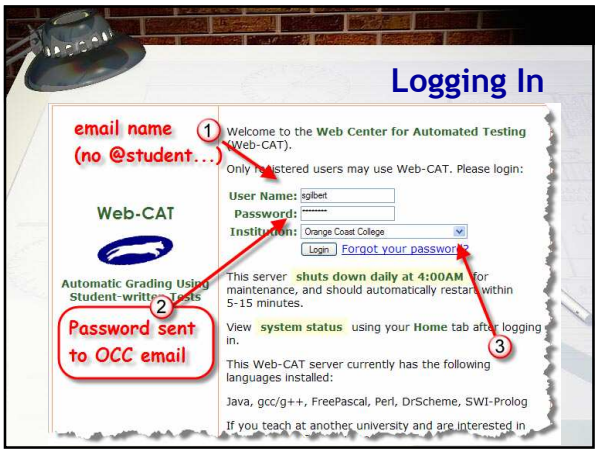
The BlueJ Submitter

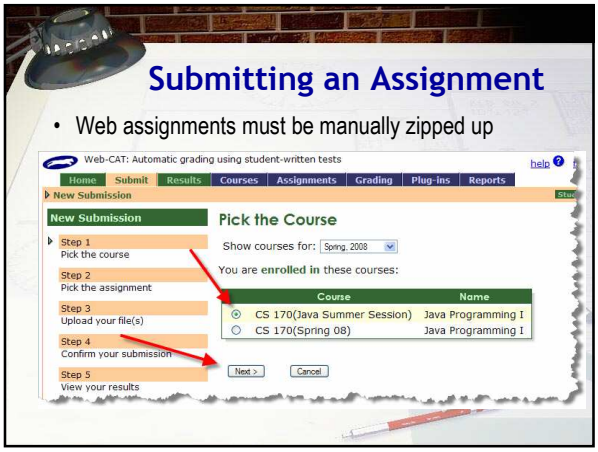
- BlueJ will automatically submit your homework
- **Step 1**: **Configure** using your OCC email name

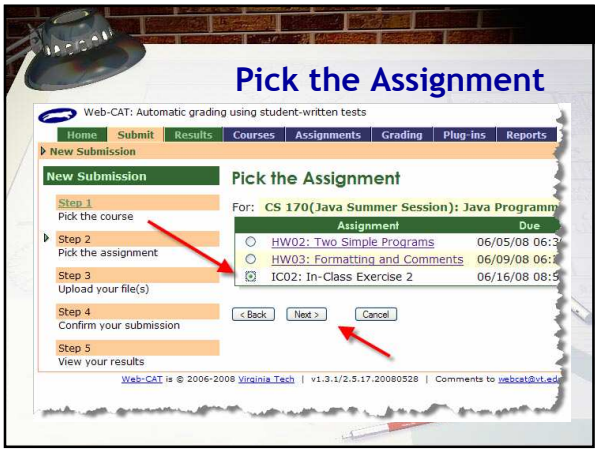


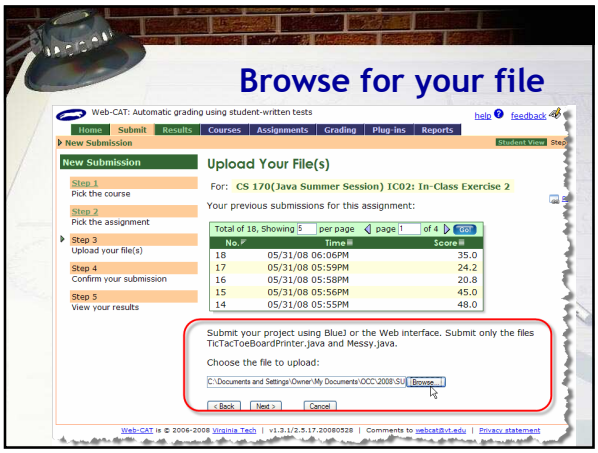
Web-Cat via the Web

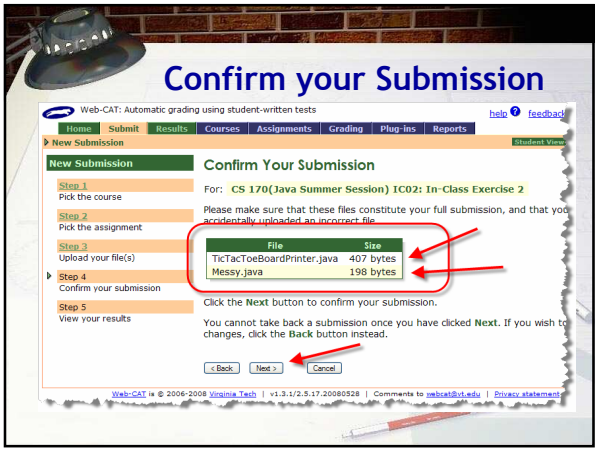
- Visit: http://web-cat.cs.vt.edu/

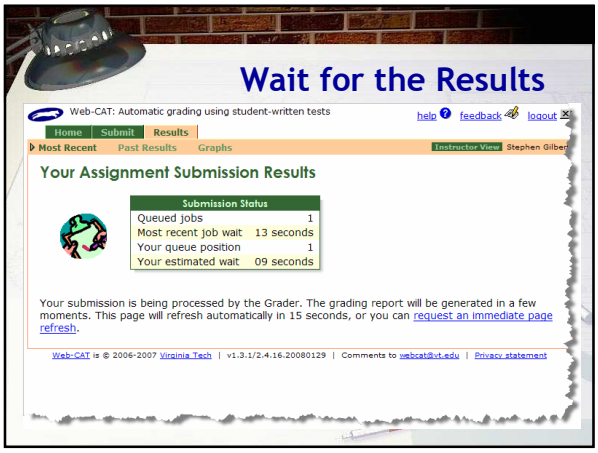

Logging In
email name
(no @student...)
1
Welcome to the Web Center for Automated Testing (Web-CAT).
Only registered users may use Web-CAT. Please login:
User Name: sgilbert
Password:
Institution: Orange Coast College
Login
Forgot your password
Web-CAT
Automatic Grading Using Student-written Tests
2
Password sent to OCC email
This server shuts down daily at 4:00AM for maintenance, and should automatically restart within 5-15 minutes.
View system status using your Home tab after logging in.
3
This Web-CAT server currently has the following languages installed:
Java, gcc/g++, FreePascal, Perl, DrScheme, SWI-Prolog
If you teach at another university and are interested in


Submitting an Assignment
• Web assignments must be manually zipped up
Web-CAT: Automatic grading using student-written tests
help
Home Submit Results Courses Assignments Grading Plug-ins Reports
New Submission
New Submission
Pick the Course
Step 1
Pick the course
Step 2
Pick the assignment
Step 3
Upload your file(s)
Step 4
Confirm your submission
Step 5
View your results
Show courses for: Spring, 2008
You are enrolled in these courses:
Course
Name
CS 170(Java Summer Session)
Java Programming I
CS 170(Spring 08)
Java Programming I
Next >
Cancel

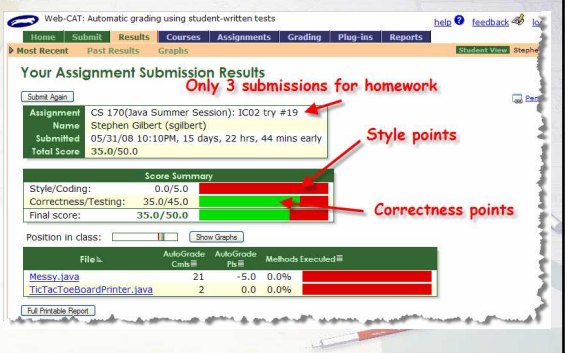

Pick the Assignment
Web-CAT: Automatic grading using student-written tests
Home Submit Results Courses Assignments Grading Plug-ins Reports
New Submission
New Submission
Pick the Assignment
Step 1
Pick the course
Step 2
Pick the assignment
Step 3
Upload your file(s)
Step 4
Confirm your submission
Step 5
View your results
For: CS 170(Java Summer Session): Java Programm
Assignment
Due
HW02: Two Simple Programs
06/05/08 06:3
HW03: Formatting and Comments
06/09/08 06:
IC02: In-Class Exercise 2
06/16/08 08:5
< Back
Next >
Cancel
Web-CAT is © 2006-2008 Virginia Tech | v1.3.1/2.5.17.20080528 | Comments to webcat@vt.edu


Browse for your file
Web-CAT: Automatic grading using student-written tests
Upload Your File(s)
For: CS 170(Java Summer Session) IC02: In-Class Exercise 2
Your previous submissions for this assignment:
Submit your project using BlueJ or the Web interface. Submit only the files TicTacToeBoardPrinter.java and Messy.java.
Choose the file to upload:
Browse...
< Back
Next >
Cancel

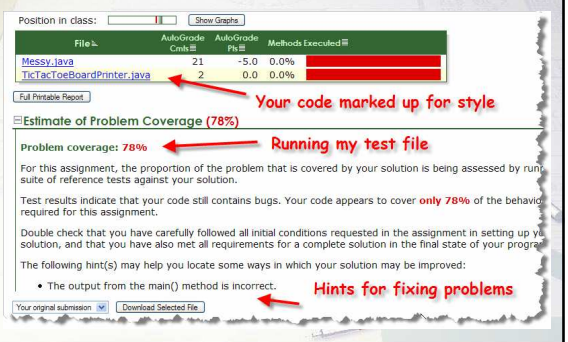

Confirm your Submission
Web-CAT: Automatic grading using student-written tests
Confirm Your Submission
For: CS 170(Java Summer Session) IC02: In-Class Exercise 2
Please make sure that these files constitute your full submission, and that you accidentally uploaded an incorrect file.
File
Size
TicTacToeBoardPrinter.java 407 bytes
Messy.java 198 bytes
Click the Next button to confirm your submission.
You cannot take back a submission once you have clicked Next. If you wish to changes, click the Back button instead.
< Back
Next >
Cancel


Wait for the Results
Web-CAT: Automatic grading using student-written tests
Your Assignment Submission Results
Submission Status
Queued jobs 1
Most recent job wait 13 seconds
Your queue position 1
Your estimated wait 09 seconds
Your submission is being processed by the Grader. The grading report will be generated in a few moments. This page will refresh automatically in 15 seconds, or you can request an immediate page refresh.

Your Results



Test Results



Lab and Homework

- In-class exercise: submit **before** you leave today!
  - Save as a PDF file
  - Drop into submissions folder on Q: drive
- Read Chapter 2 for Thursday
  - Chapter 1 quiz on Thursday
- Complete assignments on the homework page
  - Problem 2 (Thursday): similar to tonight's exercise
  - Problem 3 (Monday): style only, like Messy